Research Agent — Generated Design Rules

Summary

Total Templates: 6

Metadata Keys: domains, subdomains, templates, variable_constraints, formula_sets, impossible_conditions, mcq_distractor_rules

---

Full Design Rules (JSON Format)


```
{
  "domains" : [
    0 : "Time, Speed & Distance"
    1 : "Work & Time"
    2 : "Pipes & Cisterns"
    3 : "Profit, Loss & Discount"
    4 : "Mixtures & Ratio"
    5 : "Age-Based Problems"
  ]
  "subdomains" : {
    "Time, Speed & Distance" : [
      0 : "Linear"
      1 : "Average Speed"
      2 : "Relative Speed"
```

```
3 : "Trains"
4 : "Boats/Streams"
5 : "Circular Track"
6 : "Variable Speeds"
]
"Work & Time" : [
0 : "Combined Work"
1 : "Efficiency Change"
2 : "Joining/Leaving"
3 : "Shifts"
4 : "Breakdowns"
]
"Pipes & Cisterns" : [
0 : "Inlet/Outlet"
1 : "Leakage"
2 : "Timed Opening/Closing"
3 : "Variable Flow"
]
"Profit, Loss & Discount" : [
0 : "CP/SP/MP %"
1 : "Successive Discount"
2 : "Dishonest Shopkeeper"
3 : "Multiple Items"
4 : "GST"
5 : "Partnerships"
]
"Mixtures & Ratio" : [
0 : "Simple Mix"
```

```
1 : "Replacement"
2 : "Repeated Replacement"
3 : "Ratio Adjustment"
4 : "Alligation"
]
"Age-Based Problems" : [
0 : "Ratios"
1 : "Difference"
2 : "Past/Future"
3 : "Family Ages"
4 : "Average Age Change"
]
}
"templates" : {
"Time, Speed & Distance" : [
0 : {
"template_id" : "tsd_linear_basic"
"story_template" :
"A {object_name} travels at a constant speed of {speed_kmph} km/h. How much distance will it cover
in {time_hours} hours?"
"variables" : {
"object_name" : {
"type" : "text"
"options" : [
0 : "car"
1 : "train"
2 : "bike"
3 : "person"
]
```

```
}
"speed_kmph" : {
  "type" : "int"
  "min" : 30
  "max" : 120
}
"time_hours" : {
  "type" : "float"
  "min" : 1
  "max" : 5
  "step" : 0.5
}
}
"solution_logic" : "Distance = Speed * Time"
}
1 : {
  "template_id" : "tsd_relative_speed_opposite"
  "story_template" :
  "Two {object_type}s, {object1_name} and {object2_name}, start from two points {distance_km} km
  apart and move towards each other. {object1_name} travels at {speed1_kmph} km/h and {object2_name}
  travels at {speed2_kmph} km/h. After how many hours will they meet?"
  "variables" : {
    "object_type" : {
      "type" : "text"
      "options" : [
        0 : "cars"
        1 : "trains"
      ]
    }
    "object1_name" : {
```

```
"type" : "text"
"options" : [
  0 : "Car A"
  1 : "Train P"
]
}
"object2_name" : {
  "type" : "text"
  "options" : [
    0 : "Car B"
    1 : "Train Q"
  ]
}
"distance_km" : {
  "type" : "int"
  "min" : 100
  "max" : 500
  "constraint" : "(distance_km % (speed1_kmph + speed2_kmph) == 0)"
}
"speed1_kmph" : {
  "type" : "int"
  "min" : 40
  "max" : 80
}
"speed2_kmph" : {
  "type" : "int"
  "min" : 50
  "max" : 90
  "constraint" : "speed1_kmph != speed2_kmph"
```

```
            }
          }
          "solution_logic" : "Time = Total Distance / (Speed1 + Speed2)"
        }
      ]
      "Work & Time" : [
        0 : {
          "template_id" : "wt_combined_basic"
          "story_template" :
          "{person1_name} can complete a task in {time1_days} days. {person2_name} can complete the same task in {time2_days} days. If they work together, how many days will it take them to complete the task?"
          "variables" : {
            "person1_name" : {
              "type" : "text"
              "options" : [
                0 : "Alice"
                1 : "Bob"
              ]
            }
            "time1_days" : {
              "type" : "int"
              "min" : 10
              "max" : 30
            }
            "person2_name" : {
              "type" : "text"
              "options" : [
                0 : "Charlie"
                1 : "David"
```

```
]
}
"time2_days" : {
"type" : "int"
"min" : 12
"max" : 40
}
}
"solution_logic" : "Combined rate = (1/time1) + (1/time2). Total time = 1 / Combined rate."
}
]
"Pipes & Cisterns" : [
0 : {
"template_id" : "pc_inlet_outlet"
"story_template" :
"Pipe A can fill a tank in {timeA_hours} hours. Pipe B can empty the same tank in {timeB_hours}
hours. If both pipes are opened simultaneously, in how many hours will the tank be filled?"
"variables" : {
"timeA_hours" : {
"type" : "int"
"min" : 8
"max" : 20
}
"timeB_hours" : {
"type" : "int"
"min" : 10
"max" : 30
"constraint" : "timeB_hours > timeA_hours"
}
```

```
      }
      "solution_logic" : "Net fill rate = (1/timeA) - (1/timeB). Total time = 1 / Net fill rate."
    }
  ]
  "Profit, Loss & Discount" : [
    0 : {
      "template_id" : "pld_profit_percentage"
      "story_template" :
      "A shopkeeper buys an item for ${cost_price}. He sells it for ${selling_price}. Calculate his
      profit percentage."
      "variables" : {
        "cost_price" : {
          "type" : "int"
          "min" : 1000
          "max" : 5000
        }
        "selling_price" : {
          "type" : "int"
          "min" : 1100
          "max" : 6000
          "constraint" : "selling_price > cost_price"
        }
      }
      "solution_logic" : "Profit % = ((Selling Price - Cost Price) / Cost Price) * 100"
    }
    1 : {
      "template_id" : "pld_successive_discount"
      "story_template" :
      "An item is marked at ${marked_price}. A customer gets successive discounts of
      {discount1_percent}% and {discount2_percent}%. What is the final selling price of the item?"
```

```
"variables" : {
  "marked_price" : {
    "type" : "int"
    "min" : 2000
    "max" : 10000
    "multiples_of" : 100
  }
  "discount1_percent" : {
    "type" : "int"
    "min" : 10
    "max" : 25
    "step" : 5
  }
  "discount2_percent" : {
    "type" : "int"
    "min" : 5
    "max" : 20
    "step" : 5
    "constraint" : "discount2_percent < discount1_percent"
  }
}
"solution_logic" : "Final SP = MP * (1 - D1/100) * (1 - D2/100)"
}
]
"Mixtures & Ratio" : [
  0 : {
    "template_id" : "mr_add_water_basic"
```

```
"story_template" :
"A vessel contains {initial_volume} liters of a mixture of milk and water in the ratio {milk_ratio_initial}:{water_ratio_initial}. If {added_water} liters of water are added, what is the new ratio of milk to water?"
"variables" : {
    "initial_volume" : {
        "type" : "int"
        "min" : 50
        "max" : 150
        "constraint" : "(initial_volume % ({milk_ratio_initial} + {water_ratio_initial}) == 0)"
    }
    "milk_ratio_initial" : {
        "type" : "int"
        "min" : 2
        "max" : 5
    }
    "water_ratio_initial" : {
        "type" : "int"
        "min" : 1
        "max" : 4
    }
    "added_water" : {
        "type" : "int"
        "min" : 10
        "max" : 30
    }
}
"solution_logic" :
"Calculate initial milk and water quantities. Add 'added_water' to water quantity. Form new ratio."
```

```
      }
    ]
    "Age-Based Problems" : [
      0 : {
        "template_id" : "age_ratio_present_calc_future"
        "story_template" :
        "The present age of {person1_name} is {age1}. The present age of {person2_name} is {age2}. In how many years will the ratio of their ages be {ratio1_future}:{ratio2_future}?"
        "variables" : {
          "person1_name" : {
            "type" : "text"
            "options" : [
              0 : "Rina"
              1 : "Sam"
            ]
          }
          "person2_name" : {
            "type" : "text"
            "options" : [
              0 : "Tina"
              1 : "Uma"
            ]
          }
          "age1" : {
            "type" : "int"
            "min" : 25
            "max" : 60
          }
          "age2" : {
            "type" : "int"
```

```
"min" : 15
"max" : 40
"constraint" : "age2 < age1"
}
"ratio1_future" : {
"type" : "int"
"min" : 3
"max" : 5
}
"ratio2_future" : {
"type" : "int"
"min" : 2
"max" : 4
"constraint" : "ratio2_future < ratio1_future"
}
}
"solution_logic" :
"Let 'y' be years. Set up equation: ({age1} + y) / ({age2} + y) = {ratio1_future} /
{ratio2_future}. Solve for y."
}
]
}
"variable_constraints" : {
"Time, Speed & Distance" : {
"speed" : {
"min" : 1
"type" : "positive"
}
"time" : {
```

```
"min" : 0.1
"type" : "positive"
}
"distance" : {
"min" : 0.1
"type" : "positive"
}
}
"Work & Time" : {
"time_to_complete" : {
"min" : 1
"type" : "positive"
}
"efficiency" : {
"min" : 0.1
"type" : "positive"
}
}
"Pipes & Cisterns" : {
"fill_time" : {
"min" : 1
"type" : "positive"
}
"empty_time" : {
"min" : 1
"type" : "positive"
}
"flow_rate" : {
"min" : 0.01
```

```
      "type" : "positive"
    }
  }
  "Profit, Loss & Discount" : {
    "price" : {
      "min" : 1
      "type" : "positive"
    }
    "percentage" : {
      "min" : 0
      "max" : 100
      "type" : "range"
    }
  }
  "Mixtures & Ratio" : {
    "volume" : {
      "min" : 0.01
      "type" : "positive"
    }
    "ratio_part" : {
      "min" : 1
      "type" : "positive_integer"
    }
  }
  "Age-Based Problems" : {
    "age" : {
      "min" : 1
      "type" : "positive_integer"
    }
```

```
      "years_change" : {
        "type" : "integer"
      }
    }
  }
  "formula_sets" : {
    "Time, Speed & Distance" : [
      0 : "Distance = Speed * Time"
      1 : "Average Speed = Total Distance / Total Time"
      2 : "Relative Speed (Same Direction) = |Speed1 - Speed2|"
      3 : "Relative Speed (Opposite Direction) = Speed1 + Speed2"
    ]
    "Work & Time" : [
      0 : "Work Rate = 1 / Time"
      1 : "Total Work = Rate * Time"
      2 : "Combined Rate (A & B) = Rate_A + Rate_B"
    ]
    "Pipes & Cisterns" : [
      0 : "Net Fill Rate = (Inflow Rate - Outflow Rate)"
      1 : "Time to Fill = Total Volume / Net Fill Rate"
    ]
    "Profit, Loss & Discount" : [
      0 : "Profit = SP - CP"
      1 : "Loss = CP - SP"
      2 : "Profit % = (Profit / CP) * 100"
      3 : "Loss % = (Loss / CP) * 100"
      4 : "SP = CP * (1 + Profit%/100)"
      5 : "SP = CP * (1 - Loss%/100)"
      6 : "SP = MP * (1 - Discount%/100)"
```

7 : "Successive Discount: SP = MP * (1 - D1/100) * (1 - D2/100)"
]
"Mixtures & Ratio" : [
0 : "Ratio A:B means quantity of A / quantity of B = A/B"
1 : "Quantity of A = Total Quantity * (A / (A+B))"
2 : "Replacement: Final Quantity = Initial Quantity * (1 - (Replaced Volume / Total Volume))^n"
3 : "Alligation Rule for weighted average"
]
"Age-Based Problems" : [
0 : "Age_Future = Age_Present + Years"
1 : "Age_Past = Age_Present - Years"
2 : "Ratio Equations: (Age1 + x) / (Age2 + x) = Ratio1 / Ratio2"
]
}
"impossible_conditions" : {
"Time, Speed & Distance" : [
0 : "Negative or zero time, speed, or distance."
1 : "Object speed exceeding physical limits (e.g., speed of light for common objects)."
]
"Work & Time" : [
0 : "Negative or zero time to complete a task."
1 : "Negative work efficiency."
]
"Pipes & Cisterns" : [
0 : "Negative or zero fill/empty times."
1 :
"Total outflow rate exceeding total inflow rate when asking for tank to be filled (unless asking for time to empty)."
]

```
"Profit, Loss & Discount" : [
  0 : "Negative cost price or selling price."
  1 :
  "Percentage values outside the 0-100 range for discounts/profits/losses (unless specified otherwise
  for specific contexts like markup over 100%)."
]
"Mixtures & Ratio" : [
  0 : "Negative or zero quantities/volumes in a mixture."
  1 : "Ratios with zero or negative parts."
  2 : "Adding negative quantities of a component."
]
"Age-Based Problems" : [
  0 : "Negative or zero ages."
  1 :
  "Contradictory age statements (e.g., A is older than B, but in future, B becomes older with current
  trend)."
]
}
"mcq_distractor_rules" : [
  0 : "Arithmetic slips (e.g., addition instead of subtraction, multiplication instead of division)."
  1 : "Swapped values (e.g., using CP as SP or vice-versa, using Time1 instead of Time2)."
  2 : "Incorrect percentage conversion (e.g., using 20 instead of 0.20 for 20%)."
  3 : "Wrong ratio interpretation (e.g., A:B used as B:A)."
  4 :
  "Incorrect relative speed calculation (e.g., adding speeds for same direction, subtracting for opposite)."
  5 : "Wrong mixture replacement formula application (e.g., power 'n' incorrect, or formula inverted)."
  6 : "Calculating for the wrong variable (e.g., asking for A's age, providing B's age)."
  7 : "Ignoring specific conditions (e.g., 'after 2 hours' or 'only one pipe opened for a duration')."
  8 : "Rounding errors to a different precision level."
```

```
  ]
}
```

📥 Download design_rules.json